

TP2 — Terraform sur Azure

Infrastructure as Code — Cas fil rouge ShopEasy

Auteurs	Olivier Falahi & Paul Claverie
Formation	Mastère Dev Manager Full Stack — EFREI
Module	Optimisation du SI par l'apport du Cloud
Outil	Terraform · Provider azurearm
Cloud	Microsoft Azure (swedencentral)
Année	2025 / 2026

Projet Terraform, réponses aux ateliers, analyses FinOps & sécurité, note technique et code source.

Sommaire et conformité au rendu

Ce document unique regroupe l'ensemble du rendu du TP2. Il répond aux **livrables attendus** (section 20 du sujet).

Livrable attendu	Traité dans
1. Arborescence complète du projet Terraform	Annexe A
2. Les fichiers .tf	Annexe B (code source intégral)
3. Captures init / validate / plan / apply / output	Compte rendu §9 (checklist) — captures à produire (voir guide)
4. Capture du portail Azure (ressources)	Compte rendu §9 (checklist) — captures à produire (voir guide)
5. Capture de la page web via le Load Balancer	Compte rendu §9 (checklist) — captures à produire (voir guide)
6. Analyse de dérive (drift)	Compte rendu — Atelier 11
7. Tableaux FinOps et sécurité complétés	Analyse FinOps & sécurité
8. Note technique courte	Note technique (en tête de ce document)

Les captures d'écran (points 3, 4, 5) nécessitent un déploiement réel sur une souscription **de formation**. Le code, les analyses et les réponses sont complets ; la procédure exacte de capture est décrite dans `tp2/livrables/06_captures_a_faire.md` et les emplacements sont référencés dans la checklist technique. Dès que les images sont déposées dans `tp2/screenshots/`, elles sont intégrées automatiquement en Annexe C.

Contenu

1. Note technique (synthèse des choix + architecture)
2. Compte rendu des ateliers (1 à 14) + checklist + analyse de dérive
3. Analyse coût (FinOps), sécurité et maintenabilité
4. Mise en autonomie — Option A : subnet privé pour les données
5. Quiz de validation (20 questions)
6. Annexe A — Arborescence du projet
7. Annexe B — Code source Terraform

TP2 Terraform — Note technique

Synthèse des choix d'industrialisation de l'infrastructure ShopEasy avec Terraform sur Azure.

1. Objectif

Transformer l'architecture conçue au TP1 (réalisée manuellement via le portail) en **infrastructure déclarative**, reproductible, versionnée et destructible. Le code Terraform devient la source de vérité de l'environnement de développement ShopEasy, et la base des futurs environnements de recette et de production.

2. Architecture déployée

```
graph TD
    Internet[Internet] --> LB[Load Balancer Standard\npip-shopeasy-dev-lb]
    subgraph VNet [VNet shopeasy-dev 10.20.0.0/16]
        subgraph snetWeb [snet-web 10.20.1.0/24]
            VM1["vm-shopeasy-dev-web-1\nNginx (cloud-init)"]
            VM2["vm-shopeasy-dev-web-2\nNginx (cloud-init)"]
        end
        subgraph snetData [snet-data 10.20.2.0/24 prive]
            DB["Future base de donnees"]
        end
    end
    LB --> VM1
    LB --> VM2
    VM1 -->|"SQL 1433"| snetData
    VM2 -->|"SQL 1433"| snetData
    VM1 --> Storage[Storage Account\ncontainer documents (prive, versioning)]
    VM2 --> Storage
```

Schéma d'architecture (notation Mermaid).

Composants : 1 Resource Group, 1 VNet, 2 subnets (web + data privé), 2 NSG, 2 VM Linux Ubuntu, 3 IP publiques, 1 Load Balancer, 1 Storage Account + container privé.

3. Choix techniques

Choix	Justification
Provider <code>azurerm</code> ~> 4.0	Standard Azure, lisible, documenté ; <code>random</code> pour l'unicité du Storage
Découpage par responsabilité (<code>network</code> / <code>security</code> / <code>compute</code> / <code>loadbalancer</code> / <code>storage</code>)	Lisibilité, revue de code, limitation des conflits Git
Variables + <code>locals</code>	Aucune valeur en dur ; réutilisable multi-environnement ; tags centralisés
<code>count = 2</code> sur VM/NIC/IP	Redondance web sans duplication de code
cloud-init via <code>templatefile</code>	Provisioning automatique de Nginx, page personnalisée par serveur
Load Balancer Standard + <code>probe</code>	Suppression du SPOF web, bascule automatique sur VM saine
Storage container privé + <code>versioning</code> + TLS 1.2	Confidentialité des documents métier, protection contre suppression accidentelle
Tags <code>common_tags</code>	Gouvernance et FinOps (<code>project</code> , <code>environment</code> , <code>owner</code> , <code>managed_by</code> , <code>cost_center</code>)
Subnet <code>snet-data</code> privé (option A)	Segmentation réseau, préparation d'une couche données isolée

4. Conventions de nommage

`<type>-<projet>-<environnement>[-<role>][-<index>]` → `rg-shopeasy-dev` , `vnet-shopeasy-dev` , `snet-web` , `nsg-shopeasy-dev-web` , `vm-shopeasy-dev-web-1` , `pip-shopeasy-dev-lb` . Le Storage Account suit la contrainte Azure (minuscules/chiffres, unicité globale) : `shopeasydevdocs<suffixe-aléatoire>` .

5. Adaptations liées à la souscription Azure for Students

Contrainte	Valeur sujet	Valeur retenue	Raison
Région	<code>francecentral</code>	<code>swedencentral</code>	Policy Students interdisant <code>francecentral</code>
Gabarit VM	<code>Standard_B1s</code>	<code>Standard_B2ts_v2</code>	<code>B1s</code> indisponible (capacité) sur la souscription

Ces valeurs sont **paramétrées** (variables `location` , `vm_size`) : le passage à `francecentral` / `B1s` sur une souscription standard ne nécessite qu'un changement de `terraform.tfvars` .

6. Sécurité et gestion des secrets

- Aucun secret dans le code : seule la **clé SSH publique** est référencée par chemin (`file(var.ssh_public_key_path)`).
- `terraform.tfvars` et le **state** sont ignorés par Git (`.gitignore`).
- SSH restreint à `allowed_ssh_cidr` ; Storage privé ; subnet data sans IP publique.
- En production : backend `azurerm` distant chiffré (RBAC), Azure Key Vault, Bastion, Application Gateway + WAF.

7. Cycle de vie et reproductibilité

```
terraform init      # telecharge providers
terraform fmt       # formate
terraform validate  # verifie
terraform plan      # previsualise
terraform apply     # deploie
terraform output    # IP LB, RG, Storage
terraform destroy   # nettoie (obligatoire en fin de seance)
```

8. Limites et évolutions

Environnement volontairement minimal (pédagogique). Évolutions vers la production : suppression des IP publiques des VM + Bastion, state distant, Application Gateway + WAF, Azure SQL managé avec private endpoint dans `snet-data` , observabilité (Azure Monitor / Log Analytics), modularisation Terraform et pipeline CI/CD (`fmt / validate / plan / tfsec / infracost / approbation`).

TP2 Terraform — ShopEasy — Compte rendu des ateliers

Cas fil rouge : industrialisation de l'infrastructure **ShopEasy** sur Azure avec Terraform. Région : `swedencentral` — Resource Group : `rg-shopeasy-dev` — VNet : `10.20.0.0/16`.

Note de contexte. Comme au TP1, l'abonnement Azure for Students impose une policy limitant les régions (`francecentral` interdite) : la région retenue est `swedencentral`. Le gabarit `Standard_B1s` étant indisponible (capacité), les VM utilisent `Standard_B2ts_v2` (paramétré via la variable `vm_size`).

État du déploiement. Le code Terraform est complet et prêt à l'emploi. Les preuves d'exécution (captures terminal, portail, navigateur) sont à produire lors du `terraform apply` sur une souscription de formation, en suivant [06_captures_a_faire.md](#). Les emplacements de captures sont référencés dans la checklist (atelier 9) et l'atelier 11.

Atelier 1 — Initialiser le projet Terraform

Arborescence créée dans `tp2/terraform/` : `versions.tf`, `providers.tf`, `variables.tf`, `locals.tf`, `network.tf`, `security.tf`, `compute.tf`, `loadbalancer.tf`, `storage.tf`, `outputs.tf`, `terraform.tfvars.example`, `.gitignore` et `templates/cloud-init.yml`.

Point de contrôle — pourquoi `terraform.tfstate` ne doit pas être publié dans un dépôt non sécurisé ? Le state est la cartographie complète des ressources gérées par Terraform. Il contient des identifiants de ressources, des métadonnées et parfois des **valeurs sensibles en clair** (chaînes de connexion, clés, mots de passe générés, adresses IP, identifiants de comptes de stockage). Le publier reviendrait à exposer la topologie de l'infrastructure et des secrets exploitables par un attaquant. Il est donc exclu via `.gitignore` (et, en entreprise, stocké dans un backend distant chiffré et soumis au RBAC).

Atelier 2 — Déclarer les providers

`versions.tf` épingle `terraform >= 1.6.0`, `azurerm ~> 4.0` et `random ~> 3.6`. `providers.tf` active le provider `azurerm` avec `features {}`.

Point de contrôle — résultat attendu de `terraform validate` :

```
Success! The configuration is valid.
```

L'épinglage des versions (`required_version` , `version`) garantit la reproductibilité : un collègue ou une pipeline CI/CD utilisera exactement les mêmes versions de providers, évitant les écarts de comportement.

Atelier 3 — Paramétrer le projet

Variables d'entrée déclarées dans `variables.tf` (`project` , `environment` , `location` , `vm_size` , `admin_username` , `ssh_public_key_path` , `allowed_ssh_cidr` , et les préfixes réseau). `locals.tf` calcule `prefix = "shopeasy-dev"` et le bloc `common_tags` .

Le secret SSH n'est jamais en clair : seul le **chemin** vers la clé publique est passé (`ssh_public_key_path`), et `terraform.tfvars` (qui contient les valeurs concrètes comme `allowed_ssh_cidr`) est ignoré par Git. Un `terraform.tfvars.example` documente les valeurs attendues sans rien exposer.

Atelier 4 — Groupe de ressources et réseau

Créés dans `network.tf` : `rg-shopeasy-dev` , `vnet-shopeasy-dev` (`10.20.0.0/16`), `snet-web` (`10.20.1.0/24`) et — au titre de la mise en autonomie (option A) — `snet-data` (`10.20.2.0/24`).

Réponses aux questions (8.3) :

- Pourquoi une plage privée pour le VNet ?** Les plages `10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16` (RFC 1918) ne sont pas routables sur Internet. Elles évitent tout conflit d'adressage avec des réseaux publics, permettent de cloisonner les ressources et imposent de passer par des points d'entrée contrôlés (Load Balancer, IP publiques explicites) pour exposer un service. C'est la base de la segmentation et de la sécurité réseau.
- Que faudrait-il ajouter pour isoler une base de données dans un subnet dédié ?** Un **subnet dédié** (`snet-data` , déjà ajouté en option A), un **NSG** restrictif n'autorisant que le port de la base (1433 pour SQL) **depuis le subnet web uniquement**, et idéalement un **private endpoint** vers le service managé (Azure SQL) plus une **delegation** de subnet le cas échéant. Aucune IP publique ne doit être attachée à ce subnet.
- Pourquoi séparer les fichiers Terraform au lieu de tout mettre dans `main.tf` ?** Lisibilité et maintenabilité : chaque fichier regroupe une responsabilité (réseau, sécurité, calcul, stockage). Cela facilite les revues de code, limite les conflits Git en équipe, et permet de retrouver rapidement une ressource. Terraform charge de toute façon **tous** les `.tf` du dossier, le découpage est purement organisationnel.

Atelier 5 — Sécuriser le réseau avec un NSG

`security.tf` crée `nsg-shopeasy-dev-web` (Allow-HTTP 80 depuis Internet, Allow-SSH-Admin 22 depuis `allowed_ssh_cidr`) et son association au subnet web. L'option A ajoute `nsg-shopeasy-dev-data` (1433 depuis le subnet web + Deny-All entrant).

Analyse de sécurité (9.2)

Flux	Autorisé ?	Justification	Risque résiduel
Internet → HTTP (80)	Oui	L'application web doit être joignable publiquement via le Load Balancer	Surface exposée au niveau applicatif (DDoS L7, failles web) → atténuée par WAF/Application Gateway en prod
SSH depuis votre IP (22, <code>allowed_ssh_cidr</code>)	Oui	Administration ponctuelle restreinte à l'IP de l'apprenant	IP dynamique pouvant changer ; usurpation si poste compromis → Bastion en prod
SSH depuis Internet complet (0.0.0.0/0)	Non	Surface d'attaque massive, brute-force permanent	Aucun (règle volontairement absente)
Tout trafic sortant	Oui (par défaut Azure)	Permet <code>apt /cloud-init</code> de récupérer Nginx	Exfiltration possible si VM compromise → filtrage egress + firewall en prod

Recul (production). L'accès SSH direct serait supprimé au profit d'**Azure Bastion**, d'un VPN ou d'un jump server, voire d'une administration sans accès direct (configuration uniquement par IaC + cloud-init).

Atelier 6 — Deux machines virtuelles Linux

`compute.tf` crée, avec `count = 2` : 2 IP publiques `Standard`, 2 NIC dans `subnet-web`, 2 VM Ubuntu 22.04 (`vm_size = Standard_B2ts_v2`). Le `cloud-init.yml` installe Nginx et publie une page « ShopEasy - serveur web N » via `templatefile` (variable `server_index`).

Réponses aux questions (10.4) :

1. **À quoi sert `count` ?** À créer plusieurs instances identiques d'une ressource à partir d'un seul bloc, indexées par `count.index`. Ici, 2 VM (et leurs IP/NIC) sans dupliquer le code. Changer `count` suffit à faire varier le nombre d'instances.
2. **Rôle de `custom_data` ?** Passer un script **cloud-init** (encodé en base64) exécuté au premier démarrage de la VM. Il automatise le provisioning (installation et configuration de Nginx, page d'accueil) sans intervention manuelle : la VM est opérationnelle dès le boot.
3. **Pourquoi `Standard_B1s` (ici `B2ts_v2`) est acceptable en formation ?** Ce sont des gabarits **burstable** à très faible coût, suffisants pour héberger un Nginx de

démonstration sans charge réelle. On privilégie le coût sur la performance pour un environnement de dev/formation. (B1s indisponible sur la souscription Students → B2ts_v2 retenu.)

4. **Pourquoi éviter des IP publiques directes sur les VM en production ?** Chaque IP publique est une porte d'entrée supplémentaire à sécuriser et à superviser, augmentant la surface d'attaque. En prod, on n'expose que le Load Balancer / l'Application Gateway, on supprime les IP publiques des VM et on administre via Bastion. (Ici elles ne servent qu'à tester chaque VM individuellement.)

Atelier 7 — Load Balancer

loadbalancer.tf crée l'IP publique du LB, le LB Standard, le backend pool (associé aux 2 NIC), une probe HTTP /:80 et une règle TCP 80→80.

Réponses à l'analyse (11.3) :

1. **Quel problème résout le Load Balancer ?** Le **point de défaillance unique** (SPOF) et l'absence de répartition de charge : il distribue le trafic entrant sur plusieurs VM, améliorant la disponibilité et la capacité à absorber la charge.
2. **Que se passe-t-il si une VM devient indisponible ?** La **probe** de santé détecte l'échec (la VM ne répond plus sur /:80) et le LB cesse de lui router du trafic. Les requêtes sont automatiquement dirigées vers la VM saine restante : continuité de service (en mode dégradé).
3. **Différence avec Azure Application Gateway ?** Le Load Balancer opère en **couche 4** (TCP/UDP). L'Application Gateway opère en **couche 7** (HTTP/HTTPS) : routage par URL/host, terminaison TLS, **WAF** intégré, cookies d'affinité. L'AppGw est préférable pour exposer une application web en production ; le LB suffit pour une répartition réseau simple.

Atelier 8 — Storage Account

storage.tf crée un random_string (suffixe d'unicité globale), le Storage Account (Standard / LRS / TLS1_2 / versioning Blob activé) et un container **privé** documents.

Réponses FinOps et sécurité (12.3) :

1. **Pourquoi le container doit-il être privé ?** Il contient des documents métier (factures, données clients). Un accès public anonyme exposerait des données potentiellement sensibles et constituerait une fuite (et une non-conformité RGPD). L'accès se fait via clés/SAS/identités gérées uniquement.
2. **Pourquoi le versioning peut-il augmenter les coûts ?** Chaque modification/suppression d'un blob conserve les **versions antérieures**, qui sont facturées comme du stockage supplémentaire. Sur des objets fréquemment modifiés, le volume cumulé (et donc la facture) croît continuellement.

3. **Quelle règle de cycle de vie proposer ?** Une **lifecycle management policy** : transition des anciennes versions vers un tier froid (Cool/Archive) après N jours, puis **suppression** des versions au-delà d'une rétention définie (ex. supprimer les versions > 90 jours). Voir le détail dans `04_autonomie_subnet_prive.md` (section variante lifecycle).
-

Atelier 9 — Outputs et validation

```
outputs.tf expose resource_group_name , load_balancer_public_ip , web_vm_public_ips ,  
storage_account_name .
```

Checklist technique (13.1)

État : le code Terraform est complet et prêt. La colonne « Statut » passe à **OK** une fois `terraform apply` exécuté sur une souscription de formation et la capture correspondante déposée dans `tp2/screenshots/` (voir [06_captures_a_faire.md](#)). Tant que le déploiement n'est pas fait, le statut reste **À valider**.

Contrôle	Statut	Vérification attendue	Capture
Le projet Terraform s'initialise sans erreur	À valider	<code>terraform init</code> → Terraform has been successfully initialized!	atelier_02-init.png
<code>terraform validate</code> réussit	À valider	Success! The configuration is valid.	atelier_02-validate.png
Le Resource Group est créé	À valider	<code>terraform output resource_group_name</code> + portail	atelier_04-portail-rg.png
Le VNet et les subnets existent	À valider	<code>terraform state list</code> (<code>azurerm_virtual_network.main</code> , <code>azurerm_subnet.web</code> , <code>.data</code>)	atelier_05-vnet-subnets.png
Le NSG limite SSH à votre IP	À valider	Règle Allow-SSH-Admin source = <code>allowed_ssh_cidr</code>	atelier_05-nsg-web.png
Deux VM Linux sont créées	À valider	<code>az vm list -g rg-shopeasy-dev -o table</code> (2 lignes Running)	atelier_06-vms.png
Nginx répond sur les VM	À valider	<code>http://<IP_VM_1></code> et <code>http://<IP_VM_2></code> → page ShopEasy	atelier_06-page-vm1.png / -vm2.png
Le Load Balancer répond en HTTP	À valider	<code>http://<IP_LB></code> → page ShopEasy (alternance)	atelier_09-page-lb.png
Le Storage Account est privé	À valider	Container documents access type = <code>private</code>	atelier_08-storage.png
Le versioning Blob est activé	À valider	<code>blob_properties.versioning_enabled</code> = <code>true</code>	atelier_08-storage.png
Les ressources sont taguées	À valider	<code>common_tags</code> appliqué sur toutes les ressources taguables	atelier_04-portail-rg.png

Les captures listées sont à produire en suivant [06_captures_a_faire.md](#) ; elles apparaissent ensuite automatiquement dans l'annexe « Preuves d'exécution » du PDF de rendu.

Atelier 10 — Modifier l'infrastructure

Le tag `cost_center` = "cloud-training" est **déjà présent** dans `common_tags` (`locals.tf`). Pour l'exercice, on illustre l'ajout d'un tag supplémentaire (ex. `reviewed_by`).

Réponses à l'analyse du plan (14.2) :

1. **Terraform recrée-t-il toutes les ressources ?** Non. Un changement de tag est une modification **in-place** : Terraform affiche `~` (update) et ne touche pas au cycle de vie des ressources.

2. **Quelles ressources sont simplement mises à jour ?** Toutes les ressources taguables référençant `local.common_tags` (RG, VNet, NSG, IP, NIC, VM, LB, Storage) : seul leur bloc `tags` est mis à jour.
 3. **Pourquoi le plan est-il indispensable ?** Il permet de **vérifier l'impact avant application** : distinguer une simple mise à jour (`~`) d'une recréation destructrice (`-/+`), repérer une suppression imprévue, contrôler les coûts et la conformité. C'est le filet de sécurité du workflow déclaratif.
-

Atelier 11 — Observer une dérive (drift)

Procédure : modifier manuellement un tag sur le RG dans le portail (`manual_change = true`), puis `terraform plan`. Capture attendue : `atelier_11-drift-plan.png` (le `plan` montrant la dérive détectée).

Réponses à l'analyse (15.2) :

1. **Terraform détecte-t-il une différence ?** Oui. Au `plan`, il rafraîchit le state réel, constate l'écart entre l'infrastructure (tag manuel ajouté) et le code (qui ne le contient pas) et signale un changement.
 2. **Quelle action propose-t-il ?** De **revenir à l'état déclaré** : il prévoit de **supprimer** le tag `manual_change` ajouté à la main (`~ update in-place`), car le code est la source de vérité.
 3. **Pourquoi les modifications manuelles sont-elles dangereuses ?** Elles créent une **dérive** entre code et réel : l'infrastructure devient imprévisible, non reproductible, et les changements ne sont ni tracés ni revus. Un `apply` ultérieur peut écraser silencieusement une correction d'urgence faite à la main, ou inversement masquer un changement non documenté.
 4. **Quelle règle d'équipe proposer ? Interdire les modifications manuelles en dehors des urgences** : tout changement passe par une **pull request** modifiant le code Terraform, relue puis appliquée via pipeline. Le portail reste réservé à l'observation/diagnostic. On peut renforcer avec **Azure Policy** et des `plan` planifiés détectant le drift.
-

Atelier 12 — Préparer un state distant

Réponses aux questions (16.2) :

1. **Pourquoi le state distant facilite-t-il le travail en équipe ?** Le state est **partagé** et centralisé (un Storage Account Azure), avec **verrouillage** (lock) empêchant deux `apply` simultanés de corrompre le state. Chacun travaille sur la même source de vérité, sans s'échanger un fichier local.
2. **Pourquoi protéger l'accès au Storage Account du state ?** Parce que le state contient la cartographie et des **secrets** de l'infrastructure. Un accès non maîtrisé

permettrait fuite d'informations ou manipulation. On le protège par **RBAC**, chiffrement au repos, restriction réseau et journalisation.

3. **Pourquoi séparer le state dev / recette / prod ?** Pour **isoler les environnements** : un incident, un `destroy` ou une corruption sur le state de dev ne doit jamais impacter la prod. Cela permet aussi des droits d'accès différenciés (moindre privilège par environnement) via des `key` distinctes (`shopeasy/dev/...` , `shopeasy/prod/...`).

Atelier 14 — Nettoyage

```
terraform plan -destroy    # verifier ce qui sera detruit
terraform destroy          # supprimer toutes les ressources
```

Vérifier ensuite dans le portail que `rg-shopeasy-dev` est vide ou supprimé : aucune ressource facturable ne doit subsister.

Synthèse

L'ensemble des ateliers est couvert par le projet Terraform (`tp2/terraform/`). Les analyses FinOps et sécurité approfondies sont dans [03_analyse_finops_securite.md](#) , l'extension autonomie (option A) dans [04_autonomie_subnet_prive.md](#) , et la note technique de synthèse dans [05_note_technique.md](#) .

TP2 Terraform — Analyse coût, sécurité et maintenabilité

Atelier 13 du sujet. Cas ShopEasy, environnement de dev sur `swedencentral`.

1. Analyse FinOps (17.1)

Ressource	Coût relatif	Risque de surcoût	Optimisation proposée
VM Linux (×2)	Élevé (poste principal)	VM surdimensionnées ou laissées allumées 24/7	Gabarit burstable (<code>B2ts_v2</code>), arrêt/désallocation hors usage, auto-shutdown, <code>terraform destroy</code> en fin de séance
IP publiques (×3 : 2 VM + 1 LB)	Faible à modéré	IP Standard statiques facturées même inutilisées ; multiplication inutile	Supprimer les IP publiques des VM (accès via LB + Bastion) → ne garder que l'IP du LB
Load Balancer	Modéré (SKU Standard + règles)	Règles/IP facturées en continu	Mutualiser un seul LB pour plusieurs services ; détruire en dev hors usage
Storage Account	Faible (volume de démo)	Croissance non maîtrisée des données	Tier Standard / LRS en dev, lifecycle policy, surveiller la volumétrie
Versioning Blob	Faible mais cumulatif	Accumulation des versions antérieures facturées	Lifecycle : transition Cool/ Archive puis suppression des versions > N jours
Disques managés (OS)	Faible (<code>Standard_LRS</code>)	Disques orphelins après suppression de VM	<code>Standard_LRS</code> en dev, nettoyage des disques non rattachés, <code>destroy</code> complet

Leviers FinOps appliqués dans le projet : taille de VM adaptée (variable `vm_size`), tags de coût (`cost_center`), réplication `LRS` (la moins chère), et destruction reproductible via `terraform destroy`.

2. Analyse de sécurité (17.2)

Risque	Cause possible	Impact	Correction
SSH trop ouvert	Règle NSG en <code>0.0.0.0/0</code> sur le port 22	Brute-force, intrusion sur les VM	SSH restreint à <code>allowed_ssh_cidr (/32)</code> ; en prod, Azure Bastion / suppression du SSH public
State exposé	<code>terraform.tfstate</code> commité ou partagé sans contrôle	Fuite de secrets et de la topologie	<code>.gitignore</code> du state en local ; backend <code>azurerm</code> distant chiffré + RBAC en équipe
Storage public	<code>container_access_type</code> en <code>blob / container</code>	Fuite de documents métier (RGPD)	Container <code>private</code> (appliqué) ; accès via SAS/identités gérées uniquement
Tags absents	Ressources créées sans tags	Coûts non imputables, gouvernance impossible	<code>common_tags</code> appliqué à toutes les ressources taguables ; Azure Policy d'enforcement
Secrets dans Git	Mot de passe / clé en clair dans <code>.tf</code> ou <code>tfvars</code> versionné	Compromission d'identifiants	Aucun secret en dur ; seule la clé SSH publique est référencée par chemin ; <code>terraform.tfvars</code> ignoré ; Key Vault en prod
Modification manuelle	Changement dans le portail hors Terraform	Dérive (drift), perte de reproductibilité	Changements par PR + <code>plan / apply</code> contrôlés ; <code>plan</code> périodique de détection ; Azure Policy

3. Maintenabilité (17.3)

- Rendre le projet réutilisable pour la recette ?** Les valeurs sont déjà paramétrées par variables. Il suffit d'un jeu de valeurs par environnement : soit un fichier `recette.tfvars` (`environment = "recette"`, `vm_size`, `CIDR...`) appliqué via `terraform apply -var-file=recette.tfvars`, soit des **workspaces** Terraform, soit (mieux) un **backend** avec une `key` distincte par environnement. Le `prefix` (`shopeasy-<env>`) garantit des noms de ressources uniques par environnement.
- Quelles variables ajouter ?** `vm_count` (nombre de VM web), `os_disk_type`, `account_replication_type` (LRS/GRS selon l'env), `tags` additionnels (ex. `expiration`), un booléen `enable_public_ip_on_vm` pour couper les IP publiques en prod, et éventuellement `address_space` / préfixes déjà variabilisés.
- Quels fichiers pourraient devenir des modules ?** - `network.tf` + `security.tf` → **module** `network` (RG, VNet, subnets, NSG). - `compute.tf` → **module** `compute` (VM, NIC, IP, cloud-init). - `loadbalancer.tf` → **module** `loadbalancer`. - `storage.tf` → **module**

`storage` . Le projet racine se réduirait alors à des appels de modules paramétrés par environnement.

4. **Quelles validations automatiques en CI/CD ?** `terraform fmt -check` , `terraform validate` , `terraform plan` posté en commentaire de PR, **analyse de sécurité statique** (`tflint` , `tfsec` / `checkov`), **estimation de coût** (`infracost`), scan de secrets (gitleaks), puis `apply` gated par approbation manuelle sur les environnements sensibles.

TP2 Terraform — Mise en autonomie : Option A — Subnet privé pour les données

Atelier 19, option A du sujet. Extension de l'infrastructure ShopEasy : ajout d'un subnet privé destiné à accueillir une future base de données, sans aucune exposition publique.

1. Modification choisie

Ajouter au VNet un **subnet privé** `snet-data` (`10.20.2.0/24`) cloisonné par un **NSG dédié** `nsg-shopeasy-dev-data` :

- autorise **uniquement** le port **1433** (SQL Server) **depuis le subnet web** (`10.20.1.0/24`) ;
- **refuse tout autre trafic entrant** (règle `Deny-All-Inbound` priorité 4000) ;
- n'attache **aucune IP publique** : le subnet reste injoignable depuis Internet.

Ce choix prolonge naturellement l'architecture TP1 (qui prévoyait déjà une couche data isolée) et illustre la **segmentation réseau** (défense en profondeur) sans coût supplémentaire ni service complexe à déployer.

2. Fichiers impactés

Fichier	Changement
<code>network.tf</code>	Ajout de la ressource <code>azurerm_subnet.data</code> (<code>snet-data</code> , <code>10.20.2.0/24</code>)
<code>security.tf</code>	Ajout de <code>azurerm_network_security_group.data</code> (Allow-SQL-From-Web + Deny-All-Inbound) et de <code>azurerm_subnet_network_security_group_association.data</code>
<code>variables.tf</code>	Ajout de la variable <code>data_subnet_prefix</code> (défaut <code>10.20.2.0/24</code>)

Extrait clé (`security.tf`) :

```

resource "azurerm_network_security_group" "data" {
  name = "nsg-${local.prefix}-data"
  # ...
  security_rule {
    name                = "Allow-SQL-From-Web"
    priority             = 100
    destination_port_range = "1433"
    source_address_prefix = var.web_subnet_prefix
    # ...
  }
  security_rule {
    name                = "Deny-All-Inbound"
    priority             = 4000
    access               = "Deny"
    # ...
  }
}

```

3. Risques associés et points de vigilance

Risque	Description	Atténuation
Faux sentiment de sécurité	Le subnet est isolé, mais une base réellement déployée nécessiterait aussi un private endpoint et un chiffrement	Compléter avec Azure SQL + private endpoint + TLS lors de l'ajout réel de la base
Règle trop large	Autoriser tout le subnet web vers 1433 reste plus permissif qu'un private endpoint nominatif	Restreindre ultérieurement aux NIC précises ou via Application Security Groups
Chevauchement d'adressage	10.20.2.0/24 doit rester dans 10.20.0.0/16 et ne pas chevaucher snet-web	Préfixes gérés par variables, espaces disjoints vérifiés
Subnet vide facturé ?	Un subnet seul n'est pas facturé, mais les futures ressources le seront	Documenter ; destroy en fin de séance
Évolution vers PaaS	Pour Azure SQL managé, prévoir service delegation / private endpoint plutôt qu'une VM data	Anticipé dans la roadmap (voir note technique)

4. Variante envisagée — Option D (Storage lifecycle)

À titre complémentaire, une règle de cycle de vie limiterait le surcoût du versioning Blob (atelier 8). Elle pourrait être ajoutée dans `storage.tf` :

```
resource "azurerm_storage_management_policy" "docs" {
  storage_account_id = azurerm_storage_account.docs.id
  rule {
    name      = "expire-old-versions"
    enabled = true
    filters {
      blob_types = ["blockBlob"]
    }
    actions {
      version {
        delete_after_days_since_creation = 90
      }
      base_blob {
        tier_to_cool_after_days_since_modification_greater_than = 30
      }
    }
  }
}
```

Option A retenue comme extension principale ; la variante D est documentée comme évolution FinOps possible.

TP2 Terraform — Quiz de validation (réponses)

Réponses aux 20 questions de la section 21 du sujet TP2_Terraform_Azure.md .

1. **Terraform est-il impératif ou déclaratif ? Justifier. Déclaratif.** On décrit l'état final souhaité de l'infrastructure ; Terraform calcule lui-même la suite d'opérations (créer/modifier/détruire) nécessaire pour l'atteindre. On ne liste pas les étapes une à une.
2. **Rôle d'un provider Terraform ?** C'est un plugin qui traduit les ressources HCL en appels d'API d'une plateforme. Ici `azurerm` pilote l'API Azure Resource Manager ; `random` génère des valeurs aléatoires.
3. **À quoi sert `terraform.tfstate` ?** Il mémorise la correspondance entre les ressources déclarées dans le code et les ressources réellement créées dans le cloud (IDs, attributs). C'est la source de vérité de Terraform pour calculer les `plan` .
4. **Pourquoi `terraform plan` avant `terraform apply` ?** Pour prévisualiser les changements (créations `+` , modifications `~` , destructions `-` , remplacements `-/+`) et détecter toute action dangereuse avant d'impacter Azure.
5. **Quelle commande formate le code ?** `terraform fmt` .
6. **Quelle commande vérifie la syntaxe et la cohérence ?** `terraform validate` .
7. **Quel service Azure correspond au réseau privé logique ?** Le **Virtual Network (VNet)**.
8. **Quel composant filtre les flux réseau entrants/sortants ?** Le **Network Security Group (NSG)**.
9. **Pourquoi restreindre SSH à une seule IP ?** Pour réduire drastiquement la surface d'attaque et limiter le brute-force : seul le poste de l'administrateur peut tenter une connexion.
10. **Intérêt de `count` pour les VM ?** Déployer plusieurs instances identiques depuis un seul bloc (indexées par `count.index`), sans duplication de code, et faire varier facilement le nombre d'instances.
11. **Pourquoi utiliser des variables ?** Pour éviter les valeurs codées en dur, paramétrer le projet et le réutiliser sur plusieurs environnements (dev/test/prod) sans modifier le code.
12. **Pourquoi utiliser des outputs ?** Pour exposer après déploiement les informations utiles (IP du Load Balancer, nom du RG, nom du Storage) aux utilisateurs, scripts ou modules consommateurs.
13. **Pourquoi taguer les ressources ?** Pour la gouvernance et le FinOps : identifier le projet, l'environnement, le propriétaire et le centre de coût, faciliter le reporting des coûts et le cycle de vie.

14. **Différence entre un changement Terraform et un changement manuel dans le portail ?** Le changement Terraform est tracé (Git), revu, reproductible et applique l'état déclaré. Le changement manuel n'est pas tracé, crée une **dérive** (drift) et n'est pas reproductible.
15. **Quel risque présente un Storage Account public ?** Fuite de données : accès anonyme aux blobs (documents métier, données clients), exfiltration et non-conformité (RGPD).
16. **Pourquoi le versioning Blob peut-il générer des coûts ?** Chaque version antérieure d'un blob est conservée et facturée comme du stockage supplémentaire ; le volume cumulé croît à chaque modification.
17. **À quoi sert un Load Balancer ?** À répartir le trafic entre plusieurs VM et améliorer la disponibilité (bascule automatique via sonde de santé en cas de défaillance d'une instance).
18. **Pourquoi un state distant est-il préférable en équipe ?** State partagé et centralisé, avec verrouillage (évite les `apply` concurrents corrompant le state), sécurisable (RBAC, chiffrement) et intégrable en CI/CD.
19. **Deux bonnes pratiques de sécurité pour un projet Terraform.** (a) Ne jamais stocker de secrets dans le code/ `tfvars` versionné (Key Vault, variables d'environnement, variables CI/CD sécurisées). (b) Restreindre les accès réseau (SSH limité par IP, pas de `0.0.0.0/0`, Storage privé) et protéger le state distant par RBAC.
20. **Deux améliorations pour se rapprocher d'un environnement de production.** (a) Supprimer les IP publiques des VM et administrer via **Azure Bastion** ; externaliser le state dans un **backend Azure** sécurisé. (b) Ajouter une **Application Gateway + WAF**, un **subnet privé** pour les données, une pipeline **CI/CD** (`fmt` / `validate` / `plan` / approbation) et de l'**observabilité** (Azure Monitor / Log Analytics).

Annexe A — Arborescence du projet Terraform

```
cloud/tp2/
├─ terraform/
│   ├─ versions.tf
│   ├─ providers.tf
│   ├─ variables.tf
│   ├─ locals.tf
│   ├─ network.tf          (RG + VNet + snet-web + snet-data)
│   ├─ security.tf        (NSG web + NSG data + associations)
│   ├─ compute.tf         (2 VM Linux + NIC + IP publiques)
│   ├─ loadbalancer.tf     (LB + backend pool + probe + regle)
│   ├─ storage.tf          (Storage Account prive + container + versioning)
│   ├─ outputs.tf
│   ├─ terraform.tfvars.example
│   ├─ .gitignore
│   └─ templates/
│       └─ cloud-init.yml
├─ livrables/
│   ├─ 01_compte_rendu_ateliers.md
│   ├─ 02_quiz_reponses.md
│   ├─ 03_analyse_finops_securite.md
│   ├─ 04_autonomie_subnet_prive.md
│   ├─ 05_note_technique.md
│   └─ 06_captures_a_faire.md
└─ screenshots/           (preuves d'execution – Annexe C)
```

Annexe B — Code source Terraform

Code intégral du projet `tp2/terraform/`. Les secrets ne figurent jamais dans le code : seule la clé SSH publique est référencée par chemin, et `terraform.tfvars` est ignoré par Git.

versions.tf

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "~> 3.6"
    }
  }
}
```

providers.tf

```
provider "azurerm" {
  features {}
}
```



```

variable "project" {
  description = "Nom court du projet"
  type        = string
  default     = "shopeasy"
}

variable "environment" {
  description = "Nom de l'environnement (dev, test, prod)"
  type        = string
  default     = "dev"
}

variable "location" {
  description = "Region Azure cible. swedencentral est retenue car la policy Azure for Students interdit
  type        = string
  default     = "swedencentral"
}

variable "vm_size" {
  description = "Gabarit des VM web. Standard_B2ts_v2 est utilise car Standard_B1s est indisponible sur l
  type        = string
  default     = "Standard_B2ts_v2"
}

variable "admin_username" {
  description = "Utilisateur administrateur Linux"
  type        = string
  default     = "azureuser"
}

variable "ssh_public_key_path" {
  description = "Chemin local vers la cle publique SSH"
  type        = string
}

variable "allowed_ssh_cidr" {
  description = "CIDR autorise pour SSH (idealement l'IP publique de l'apprenant en /32)"
  type        = string
}

variable "vnet_address_space" {
  description = "Espace d'adressage du Virtual Network"
  type        = string
  default     = "10.20.0.0/16"
}

```

```
variable "web_subnet_prefix" {
  description = "Prefixe du subnet applicatif web"
  type        = string
  default     = "10.20.1.0/24"
}

variable "data_subnet_prefix" {
  description = "Prefixe du subnet prive de donnees (mise en autonomie - option A)"
  type        = string
  default     = "10.20.2.0/24"
}
```

locals.tf

```
locals {
  prefix = "${var.project}-${var.environment}"

  common_tags = {
    project      = var.project
    environment  = var.environment
    owner        = "formation"
    managed_by   = "terraform"
    cost_center  = "cloud-training"
  }
}
```

network.tf

```
resource "azurerm_resource_group" "main" {
  name      = "rg-${local.prefix}"
  location  = var.location
  tags      = local.common_tags
}

resource "azurerm_virtual_network" "main" {
  name                = "vnet-${local.prefix}"
  address_space       = [var.vnet_address_space]
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags                = local.common_tags
}

resource "azurerm_subnet" "web" {
  name                = "snet-web"
  resource_group_name = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = [var.web_subnet_prefix]
}

# Mise en autonomie - Option A : subnet prive destine a une future base de donnees.
# Il n'expose aucune ressource publique et est filtre par nsg-data (voir security.tf).
resource "azurerm_subnet" "data" {
  name                = "snet-data"
  resource_group_name = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = [var.data_subnet_prefix]
}
```



```

resource "azurerm_network_security_group" "web" {
  name            = "nsg-${local.prefix}-web"
  location        = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags            = local.common_tags

  security_rule {
    name            = "Allow-HTTP"
    priority        = 100
    direction       = "Inbound"
    access          = "Allow"
    protocol        = "Tcp"
    source_port_range = "*"
    destination_port_range = "80"
    source_address_prefix = "Internet"
    destination_address_prefix = "*"
  }

  security_rule {
    name            = "Allow-SSH-Admin"
    priority        = 110
    direction       = "Inbound"
    access          = "Allow"
    protocol        = "Tcp"
    source_port_range = "*"
    destination_port_range = "22"
    source_address_prefix = var.allowed_ssh_cidr
    destination_address_prefix = "*"
  }
}

resource "azurerm_subnet_network_security_group_association" "web" {
  subnet_id            = azurerm_subnet.web.id
  network_security_group_id = azurerm_network_security_group.web.id
}

# Mise en autonomie - Option A : NSG du subnet prive de donnees.
# Seul le port 1433 (SQL) est autorise et uniquement depuis le subnet web.
# Aucun acces entrant depuis Internet : le subnet data reste prive.
resource "azurerm_network_security_group" "data" {
  name            = "nsg-${local.prefix}-data"
  location        = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags            = local.common_tags

```

```

security_rule {
  name           = "Allow-SQL-From-Web"
  priority       = 100
  direction      = "Inbound"
  access         = "Allow"
  protocol       = "Tcp"
  source_port_range = "*"
  destination_port_range = "1433"
  source_address_prefix = var.web_subnet_prefix
  destination_address_prefix = "*"
}

security_rule {
  name           = "Deny-All-Inbound"
  priority       = 4000
  direction      = "Inbound"
  access         = "Deny"
  protocol       = "*"
  source_port_range = "*"
  destination_port_range = "*"
  source_address_prefix = "*"
  destination_address_prefix = "*"
}

}

resource "azurerm_subnet_network_security_group_association" "data" {
  subnet_id          = azurerm_subnet.data.id
  network_security_group_id = azurerm_network_security_group.data.id
}

```



```

resource "azurerm_public_ip" "web" {
  count          = 2
  name           = "pip-${local.prefix}-web-${count.index + 1}"
  location       = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method = "Static"
  sku            = "Standard"
  tags           = local.common_tags
}

resource "azurerm_network_interface" "web" {
  count          = 2
  name           = "nic-${local.prefix}-web-${count.index + 1}"
  location       = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags           = local.common_tags

  ip_configuration {
    name                  = "internal"
    subnet_id             = azurerm_subnet.web.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id   = azurerm_public_ip.web[count.index].id
  }
}

resource "azurerm_linux_virtual_machine" "web" {
  count          = 2
  name           = "vm-${local.prefix}-web-${count.index + 1}"
  location       = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  size           = var.vm_size
  admin_username = var.admin_username
  network_interface_ids = [azurerm_network_interface.web[count.index].id]

  admin_ssh_key {
    username = var.admin_username
    public_key = file(var.ssh_public_key_path)
  }

  os_disk {
    caching          = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {

```



```
    publisher = "Canonical"
    offer      = "0001-com-ubuntu-server-jammy"
    sku        = "22_04-lts"
    version    = "latest"
  }

  custom_data = base64encode(templatefile("${path.module}/templates/cloud-init.yml", {
    server_index = count.index + 1
  }))

  tags = local.common_tags
}
```



```

resource "azurerm_public_ip" "lb" {
  name            = "pip-${local.prefix}-lb"
  location        = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method = "Static"
  sku             = "Standard"
  tags            = local.common_tags
}

resource "azurerm_lb" "web" {
  name            = "lb-${local.prefix}-web"
  location        = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  sku             = "Standard"
  tags            = local.common_tags

  frontend_ip_configuration {
    name            = "public-frontend"
    public_ip_address_id = azurerm_public_ip.lb.id
  }
}

resource "azurerm_lb_backend_address_pool" "web" {
  name            = "backend-web"
  loadbalancer_id = azurerm_lb.web.id
}

resource "azurerm_network_interface_backend_address_pool_association" "web" {
  count            = 2
  network_interface_id = azurerm_network_interface.web[count.index].id
  ip_configuration_name = "internal"
  backend_address_pool_id = azurerm_lb_backend_address_pool.web.id
}

resource "azurerm_lb_probe" "http" {
  name            = "http-probe"
  loadbalancer_id = azurerm_lb.web.id
  protocol        = "Http"
  request_path     = "/"
  port            = 80
}

resource "azurerm_lb_rule" "http" {
  name            = "http-rule"
  loadbalancer_id = azurerm_lb.web.id
}

```

```

protocol                = "Tcp"
frontend_port           = 80
backend_port            = 80
frontend_ip_configuration_name = "public-frontend"
backend_address_pool_ids = [azurerm_lb_backend_address_pool.web.id]
probe_id                = azurerm_lb_probe.http.id
}

```

storage.tf

```

# Les noms de Storage Account doivent etre globalement uniques (3-24 caracteres, minuscules/chiffres).
resource "random_string" "suffix" {
  length  = 6
  upper   = false
  special = false
}

resource "azurerm_storage_account" "docs" {
  name                        = "${replace(local.prefix, "-", "")}docs${random_string.suffix.result}"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  account_tier               = "Standard"
  account_replication_type   = "LRS"
  min_tls_version            = "TLS1_2"
  tags                       = local.common_tags

  blob_properties {
    versioning_enabled = true
  }
}

resource "azurerm_storage_container" "documents" {
  name                = "documents"
  storage_account_id = azurerm_storage_account.docs.id
  container_access_type = "private"
}

```

outputs.tf

```
output "resource_group_name" {
  description = "Nom du Resource Group cree"
  value       = azurerm_resource_group.main.name
}

output "load_balancer_public_ip" {
  description = "Adresse IP publique du Load Balancer (point d'entree de l'application)"
  value       = azurerm_public_ip.lb.ip_address
}

output "web_vm_public_ips" {
  description = "Adresses IP publiques directes des deux VM web"
  value       = azurerm_public_ip.web[*].ip_address
}

output "storage_account_name" {
  description = "Nom du Storage Account des documents"
  value       = azurerm_storage_account.docs.name
}
```

terraform.tfvars.example

```
# Copier ce fichier en terraform.tfvars puis adapter les valeurs.
# terraform.tfvars est ignore par Git (peut contenir des informations propres a l'environnement).

project           = "shopeasy"
environment       = "dev"
location          = "swedencentral"
vm_size           = "Standard_B2ts_v2"
admin_username    = "azureuser"
ssh_public_key_path = "~/ssh/id_rsa.pub"

# Remplacer par votre IP publique en /32 (ex: curl ifconfig.me) - ne jamais mettre 0.0.0.0/0.
allowed_ssh_cidr = "X.X.X.X/32"
```

templates/cloud-init.yml

```
#cloud-config
package_update: true
packages:
  - nginx
write_files:
  - path: /var/www/html/index.html
    content: |
      <html>
      <body>
      <h1>ShopEasy - serveur web ${server_index}</h1>
      <p>Deploiement realise avec Terraform sur Azure.</p>
      </body>
      </html>
runcmd:
  - systemctl enable nginx
  - systemctl restart nginx
```